

Cetus

Security Assessment

May 16th, 2024 — Prepared by OtterSec

Michał Bochnak embe221ed@osec.io

Sangsoo Kang sangsoo@osec.io

Robert Chen notdeghost@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

Findings **4**

Vulnerabilities **5**

OS-CLO-ADV-00 | Loss Of Coin 6

OS-CLO-ADV-01 | Unsafe Role Removal 7

OS-CLO-ADV-02 | Incorrect Whitelist Check 8

General Findings **9**

OS-CLO-SUG-00 | Code Refactoring 10

Appendices

Vulnerability Rating Scale **11**

Procedure **12**

© 2024 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Cetus Protocol engaged OtterSec to assess the `limit-order` program. This assessment was conducted between May 8th and May 16th, 2024. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 4 findings throughout this audit engagement.

In particular, we identified a critical vulnerability allowing the loss of coins in limit orders(OS-CLO-ADV-00), unsafe role removal in access control (OS-CLO-ADV-01), and incorrect implementation of minimum trade amount checks for whitelisted tokens (OS-CLO-ADV-02).

We also recommended adding several validations into the code (OS-CLO-SUG-00).

© 2024 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/CetusProtocol/cetus-limitorder. This audit was performed against commit 53bff8b.

A brief description of the programs is as follows:

Name	Description
limit-order	The limit order module on Sui places limit orders at specific rate levels, independently seeks matching trade paths, and supports various trading strategies and asset types.

© 2024 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 4 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	1
HIGH	0
MEDIUM	1
LOW	1
INFO	1

© 2024 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-CLO-ADV-00	CRITICAL	RESOLVED	Lack of validation in `repay_flash_loan` allows users to steal coins.
OS-CLO-ADV-01	MEDIUM	RESOLVED	Role removal using the subtraction operation may affect other roles.
OS-CLO-ADV-02	LOW	RESOLVED	`config::check_token_and_min_trade_amount` incorrectly checks the minimum trade amount when the token is in the whitelist.

© 2024 Otter Audits LLC. All Rights Reserved.

Loss Of Coin CRITICAL OS-CLO-ADV-00

Description

`limit_order::repay_flash_loan` lacks a check to ensure that the `order_id` in `receipt` matches `limit_order`. This omission allows a user to deposit `target_coin` into an order different from the one that initiated the `limit_order::flash_loan`. Consequently, the original order loses `PayCoin` and fails to receive `TargetCoin`, resulting in a financial loss.

Proof of Concept

1. An attacker creates an order with the same `PayCoin` and `TargetCoin` as the victim's order.
2. The attacker flash loans the victim's order to take the `PayCoin`.
3. When repaying the flash loan, the attacker deposits the amount of `TargetCoin` specified in the receipt into their own order instead of the victim's order.
4. The victim does not receive the `TargetCoin`, and the attacker claims the `TargetCoin` from their own order.

Remediation

Ensure that the `order_id` in the `receipt` matches the `ID` of the `limit_order`.

>_ order.move diff

```
@@ -677,6 +677,8 @@ module limit_order::limit_order {
             target_repay_amount
         } = receipt;

+        assert!(order_id == id(limit_order), EMismatchedOrder);
+
         // store target coin into order
         let target_coin = coin::split(coin, target_repay_amount, ctx);
         let target_balance = coin::into_balance(target_coin);
```

Patch

Fixed in aa1eba1.

© 2024 Otter Audits LLC. All Rights Reserved.

Unsafe Role Removal MEDIUM OS-CLO-ADV-01

Description

In `acl::remove_role`, utilizing the subtraction operation for role removal may produce unexpected results. The current implementation fails to check if the role to be removed is set, allowing attempts to remove an unset role, which may result in other roles being added or removed. Employ a safe operation to guarantee that only the specified role is removed.

>_ acl.move rust

```rust
    /// @notice Revoke a role for a member in the ACL.
    public fun remove_role(acl: &mut ACL, member: address, role: u8) {
        assert!(role < 128, ERoleNumberTooLarge);
        if (linked_table::contains(&acl.permissions, member)) {
            let perms = linked_table::borrow_mut(&mut acl.permissions, member);
            *perms = *perms - (1 << role);
        }
    }
```

Remediation

Replace the operation with `*perms = *perms & (MAX_U128 - (1 << role))`. In this case, the `and` operation sets the only bit position indicated by the `role` to zero.

Patch

Fixed in aa1eba1.

© 2024 Otter Audits LLC. All Rights Reserved.

Incorrect Whitelist Check LOW OS-CLO-ADV-02

Description

`config::check_token_and_min_trade_amount` misuses the return value of `find_token_and_min_trade_amount` when `require_check_token_white_list` is true. The `is_existed` indicates the presence of the token in the whitelist. If it is true, the function should verify the minimum trade amount. Conversely, if `is_existed` is false, meaning the token is absent from the whitelist, the function should immediately return false.

>_ config.move rust

```
public fun check_token_and_min_trade_amount<Coin>(config: &GlobalConfig, amount: u64): bool {
    if (!config.require_check_token_white_list) {
        true
    } else {
        let (is_existed, min_trade_amount) = find_token_and_min_trade_amount<Coin>(config);
        if (!is_existed && min_trade_amount <= amount) {
            true
        } else {
            false
        }
    }
}
```

Remediation

Change the condition to `is_existed && min_trade_amount <= amount`.

Patch

Fixed in aa1eba1.

© 2024 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-CLO-SUG-00	Recommendations for code base modifications.

© 2024 Otter Audits LLC. All Rights Reserved.

Code Refactoring OS-CLO-SUG-00

Description

1. Prevent the creation of expired orders and zero balance orders in `place_limit_order`.
2. Add the same validation present in `place_limit_order` to `create_indexer_and_place_limit_order`.
3. In `update_min_trade_amount`, skip emitting an event if no update occurs and require the manager role instead of `adminCap` to maintain consistency with `add_into_token_list` and `remove_from_token_list`.
4. When `config.require_check_token_white_list` is `true`, verify in `flash_loan` that the `targetCoin` type is in the whitelist.

Remediation

1. Ensure `expired_ts` is greater than `created_ts` and `pay_coin` is greater than zero.
2. Call `place_limit_order` after creating an indexer.
3. Replace `adminCap` with a call to `checked_token_white_list_manager_role` and place the emit code inside an if statement.
4. Call `checked_pool_token_types` that verifies `PayCoin` and `TargetCoin` are in the whitelist.

Patch

Fixed in aa1eba1 and ed9ecf5.

© 2024 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2024 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that the other missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2024 Otter Audits LLC. All Rights Reserved.